Morris Traversal

: In Binary Tree, it can go through ascending order with Space Complexity O(1)

How to

① Find my deepest right child

② set up a pointer from the child to myself

③ Once we visit the node again and find the rightmost child.
if it has set up returning point already, then time to move!

Logic summary.

① if no left child, move right child

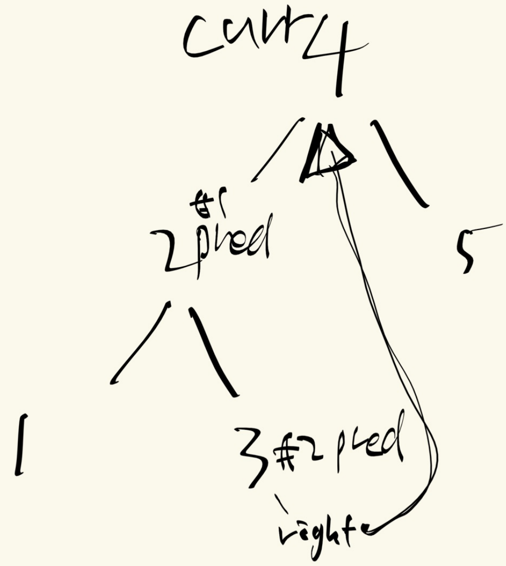
② if left exists, find its rightmost child, and set up a pointer to myself
a Link

③ if left exists, and it has the Link, remove the Link and move

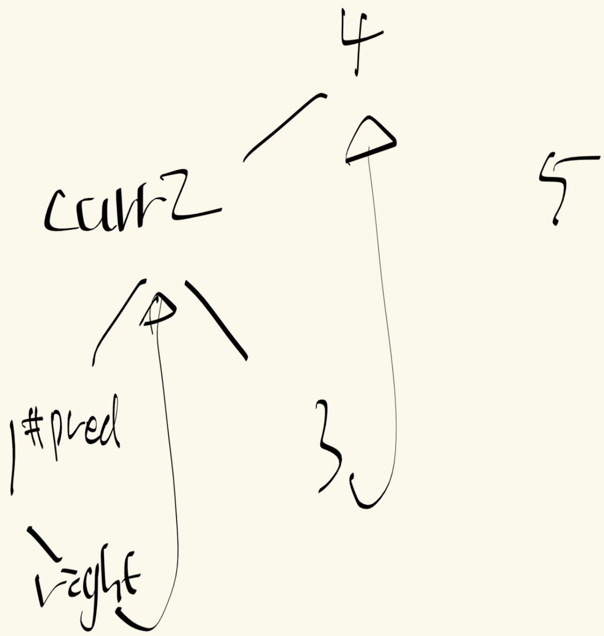
Code (Inorder)

```
res = []
curr = root

while curr:
    if curr.left is None:
        # visit the node
        res.append(curr.val)
        curr = curr.right

    else:
        predecessor = curr.left
        while predecessor.right and predecessor.right != curr:
            predecessor = predecessor.right


        if predecessor.right is None:
            # Preorder, visit value here
            # res.append(curr.val)
            predecessor.right = curr
            curr = curr.left
        else:
            # I arrive this node by the Link
            res.append(curr.val)
            curr = curr.right

    return res
```

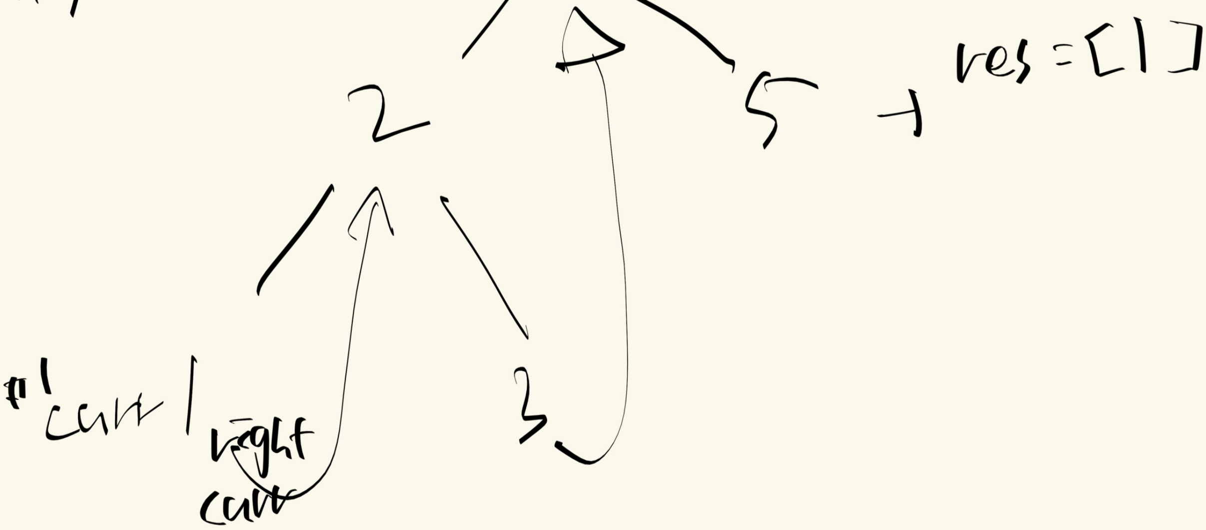
#1


curr 4
#1
2 pred
5
1
3 #2 pred
right

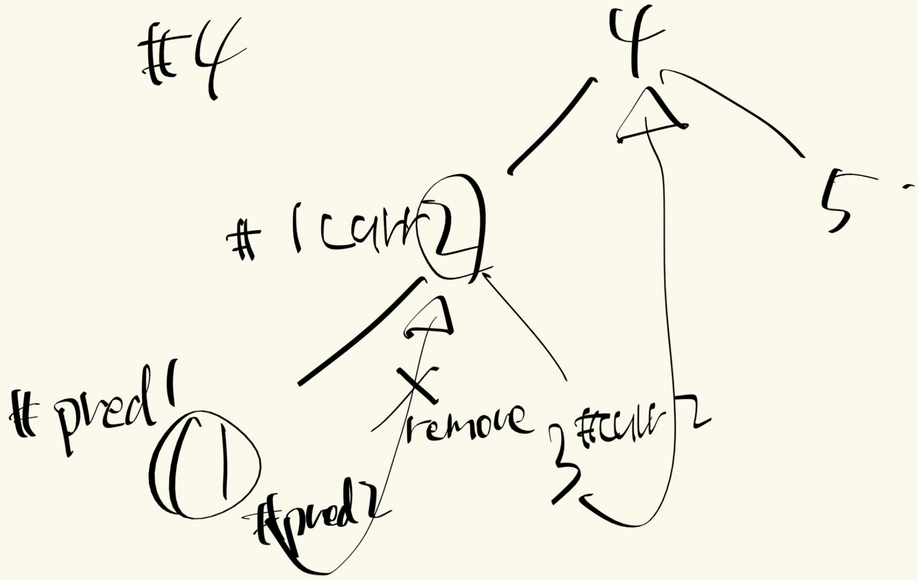
#2


4
curr 2
5
1 #pred
3
right

#3


4
2
5
res = [1]
#1 curr 1
right
curr
3

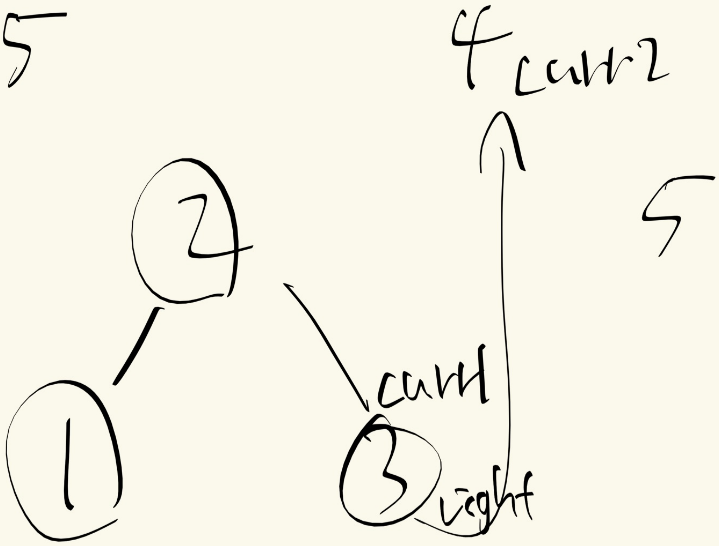
#4



→ res = [1, 2]

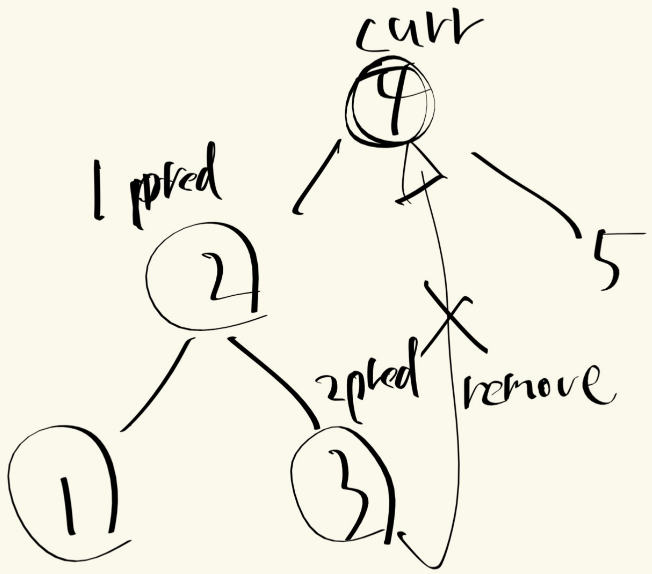
curr2 == pred1.right

#5



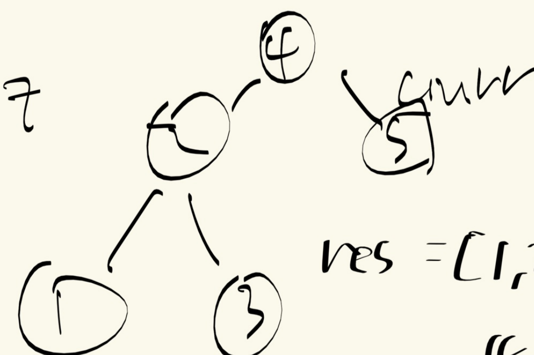
→ res = [1, 2, 3]

#6



→ res = [1, 2, 3, 4]

#7



res = [1, 2, 3,
4, 5]